

Teste Técnico Nodejs

Este documento descreve o escopo para o desenvolvimento de uma API REST que gerencia o ciclo de vida de cobranças de um sistema financeiro, com integração específica para processamento de pagamentos via **PIX** e **Cartão de Crédito**.

1. Funcionalidades da API

A API deve oferecer os seguintes endpoints para a gestão de pagamentos:

- **Adicionar Pagamento:** `POST /api/payment`
 - Cria um novo registro de pagamento no sistema.
- **Atualizar Pagamento:** `PUT /api/payment/{id}`
 - Permite a atualização dos dados de um pagamento existente, como o status.
- **Buscar Pagamento por ID:** `GET /api/payment/{id}`
 - Retorna os detalhes de um pagamento específico.
- **Listar Pagamentos:** `GET /api/payment`
 - Permite buscar pagamentos com base em filtros, como **CPF** e **meio de pagamento**.

2. Estrutura do Domínio de Pagamento

O modelo de dados para o pagamento deve incluir as seguintes propriedades:

- `id`: Identificador único do pagamento.
- `cpf`: CPF do cliente.
- `description`: Descrição da cobrança.
- `amount`: Valor da transação.
- `paymentMethod`: Define o meio de pagamento (e.g., `'PIX'` ou `'CREDIT_CARD'`).
- `status`: O estado atual do pagamento. Os valores possíveis são:
 - `PENDING`: Pagamento pendente.
 - `PAID`: Pagamento aprovado com sucesso.
 - `FAIL`: Erro no processamento da transação.

3. Regras de Negócio e Integrações

As seguintes regras devem ser implementadas para o processamento das transações:

Pagamentos via PIX

- Quando a `paymentMethod` for **PIX**, o sistema deve apenas criar o registro do pagamento no banco de dados com o status **PENDING**. Não será necessária uma integração externa para este tipo de transação na etapa inicial.

Pagamentos via Cartão de Crédito

- Quando a `paymentMethod` for **CREDIT_CARD**, a API deve **obrigatoriamente** se integrar com a API do **Mercado Pago** para processar a transação.
- Deve ser utilizada a [API de Preferências do Checkout do Mercado Pago](#) para iniciar o processo de pagamento.
- A API deve estar preparada para receber o **callback** (notificação) do Mercado Pago, que será utilizado para **atualizar o status** do pagamento no nosso sistema (por exemplo, de **PENDING** para **PAID** ou **FAIL**).

4. Considerações Técnicas

- Implementar testes unitários.
- A API deve seguir o padrão RESTful.
- A implementação de validações para os dados de entrada (**CPF**, **amount**, etc.) é fundamental para a integridade do sistema.
- Utilizar Clean Architecture.
- Utilizar controle de versão.
- **Opcional**: Caso necessário utilize **Temporal.io** para criar workflow e orquestrar chamadas para serviços do mercado pago.
 - O **Temporal.io** será a espinha dorsal para garantir que as operações de pagamento com **Cartão de Crédito** sejam robustas e confiáveis. Em vez de depender de um *callback* simples, você criará um *workflow* para gerenciar toda a transação:

Este *workflow* se encarregará de:

- Registrar o pagamento no banco de dados com o status **PENDING**.
- Chamar o serviço de integração com o Mercado Pago para criar a transação.
- **Aguardar o retorno do Mercado Pago (via *callback* ou *polling*) de forma durável**. Se o servidor cair, o *workflow* não será perdido; ele continuará de onde parou.
- Atualizar o status do pagamento para **PAID** ou **FAIL** no banco de dados, com base na resposta do Mercado Pago.

5. Tecnologias sugeridas

- <https://nestjs.com/>
- <https://www.nodejs.tech/pt-br>
- <https://www.postgresql.org/>
- <https://temporal.io/>

6. Link do repositório

- Enviar o link com repositório para os emails abaixo:
 - pedro.costa@exa.com.br
 - alvaro.souza@exa.com.br
 - capitani15@exa.com.br